

ALU 设计

1. 指定的指令如下

指令	类型	opcode	function	含义
addu	R	00000	100001	将寄存器 rs 的值与寄存器 rt 的值相加，结果写入 rd 寄存器中。
sub	R	00000	100010	将寄存器 rs 的值与寄存器 rt 的值相减，结果写入 rd 寄存器中。不考虑溢出。
slt	R	00000	101010	将寄存器 rs 的值与寄存器 rt 中的值进行有符号数比较，如果寄存器 rs 中的值小，则寄存器 rd 置 1；否则寄存器 rd 置 0。实现的时候可以实现位 rs-rt, 将符号位送 rd 的第 0 位。
lui	I	001111	无	将 16 位立即数 imm 写入寄存器 rt 的高 16 位，寄存器 rt 的低 16 位置 0。
ori	I	001101	无	寄存器 rs 中的值与 0 扩展至 32 位的立即数 imm 按位逻辑或，结果写入寄存器 rt 中。
bne	I	000101	无	如果寄存器 rs 的值不等于寄存器 rt 的值则转移，否则顺序执行。转移目标由立即数 offset 左移 2 位并进行有符号扩展的值加上该分支指令对应的延迟槽指令的 PC 计算得到。实现的时候可以用 ALU 做减法，然后用是否为 0 标志位进行控制。
lw	I	100011	无	将 base 寄存器（rs 的位置）的值加上符号扩展后的立即数 offset 得到访存的地址，据此地址从存储器中读取连续 4 个字节的值，写入到 rt 寄存器中。注意：测试程序中给出的地址是对齐的，所以不用考虑地址不对齐触发例外。
sw	I	101011	无	将 base 寄存器的值加上符号扩展后的立即数 offset 得到访存的虚地址，如果地址不是 4 的整数倍则触发地址错例外，否则据此虚地址将 rt 寄存器存入存储器中。注意：测试程序中给出的地址是对齐的，所以不用考虑地址不对齐触发例外。
stp	其他	11111	无	所有控制信号清 0，时钟输入终止。如果是五段流水，要求 stp 指令也为五段，也就是 stp 指令退出流水线后，才能停止时钟输入。

2. 根据指令，对 ALU 的功能进行总结。如果某指令不需要 alu，可以填写“无”。

指令功能总结	功 能	说明
addu	无符号加法	计算两个操作数的无符号和，结果写入目标寄存器
sub	减法	计算两个操作数的差（源操作数 1 - 源操作数 2），结果写入目标寄存器
slt	小于则置位	比较两个操作数，若源操作数 1 < 源操作数 2，结果为 1；否则为 0，写入目标寄存器
lui	高位加载	将立即数左移 16 位后写入目标寄存器的高 16 位，低 16 位补 0
ori	按位或运算	操作数与立即数进行按位或运算，结果写入目标寄存器
bne	相等则分支	比较两个操作数是否不相等，不相等则跳转（需 ALU 执行减法 / 比较操作）
lw	加法	从内存加载数据到寄存器，涉及地址计算
sw	加法	将寄存器数据存储到内存，涉及地址计算
stp	无	停机指令，无运算操作

3. 根据上述归纳总结的功能，设计 ALU 的控制码，可以采用独热码，也可以采用二进制编码。

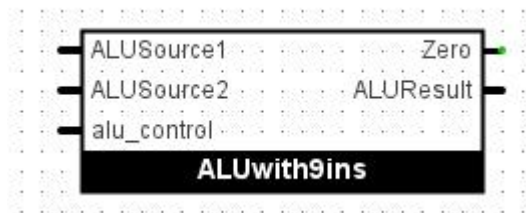
功能	编码
无符号加法	000
减法	001
逻辑或	010
小于则置位	011
高位加载	100

4. 完成 ALU 模块的设计。

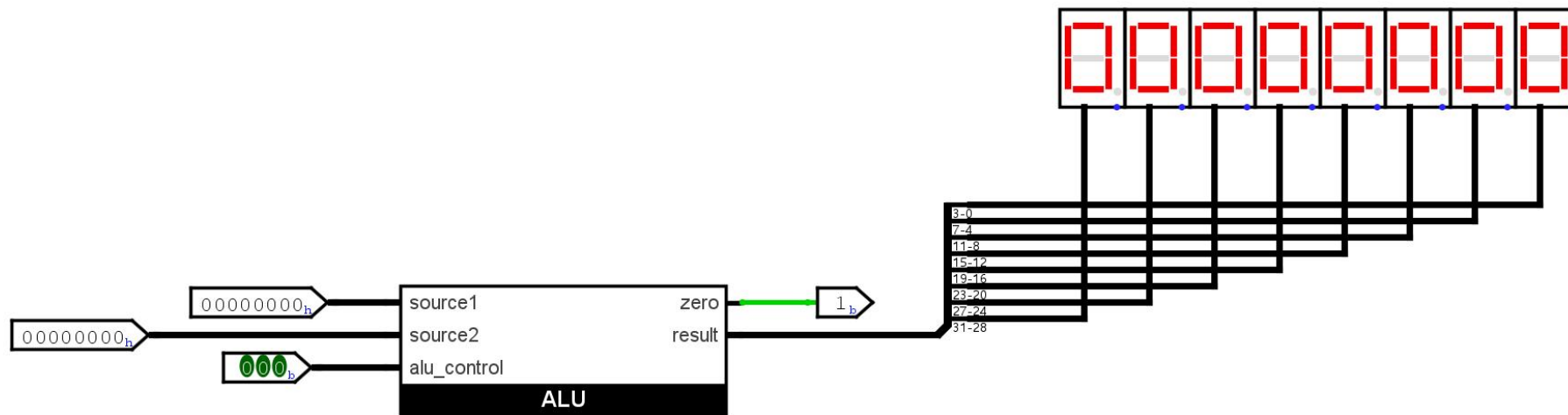
第一步，定义了一个 ALUwith9ins 的模块，先把输入输出定义好，如：

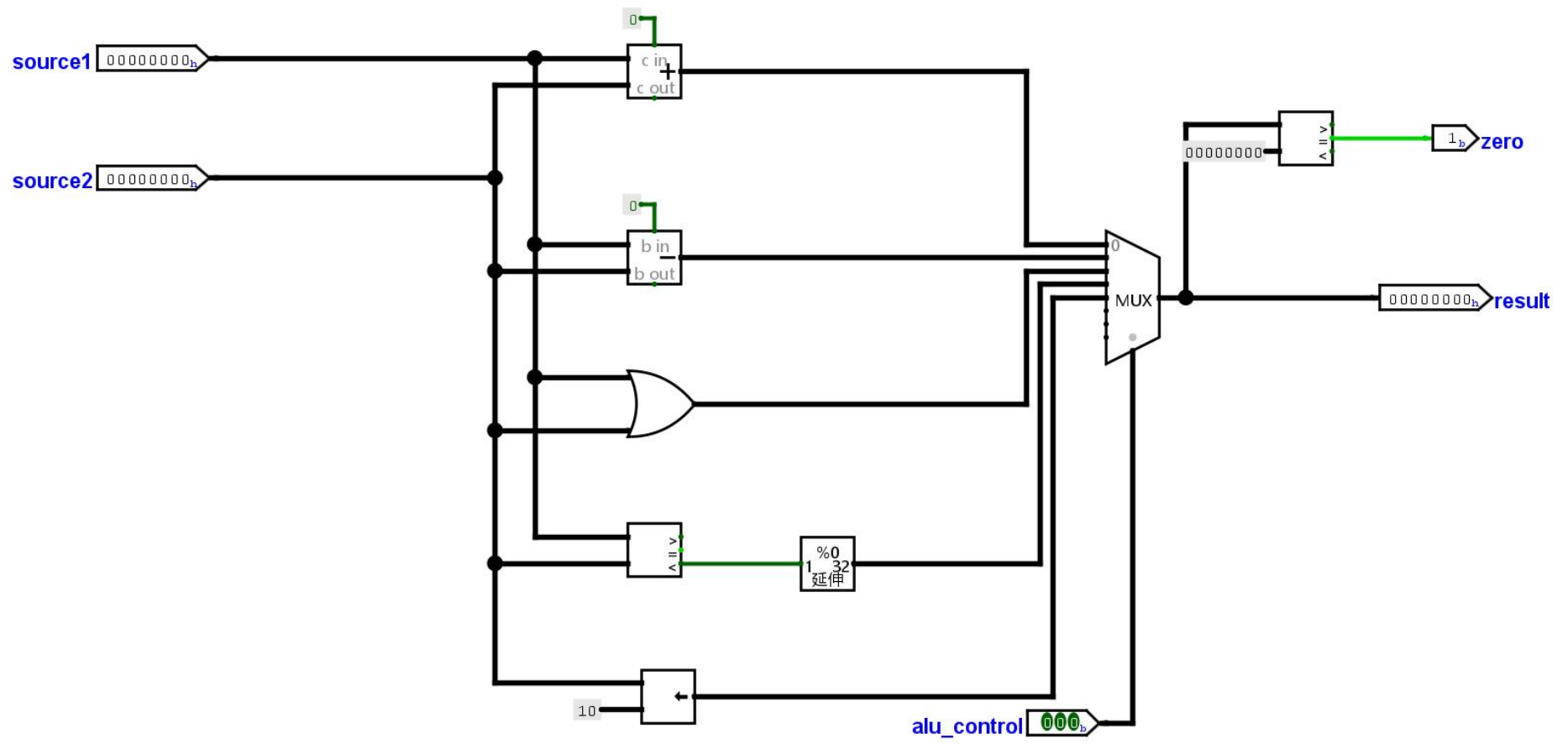
输入	输出
ALUSource1 (32 位) ALUSource2 (32 位)	ALUResult (32 位) Zero (1 位。ALUResult 为 0 时，输出 1)

在 main 中，把这个模块拖过来，就是这个样子：



第二步，按照任务二总结的功能，完成 ALUwith9ins 模块，截图如下：





5. 请将各个功能测试的截图列出。

alu

main

ALU

线路

逻辑门

复用器

运算器

存储库

输入/输出

TTL

TCL

BFH mega 函数

额外输入/输出

SOC

属性

状态

引脚 (240,280)

是否支持 FPGA:

支持

方向

→东

输出?

否

数据位

32

三态?

否

拉动行为

不变

标签

HDL 所需

标签字体

SansSerif 粗体 16

基数

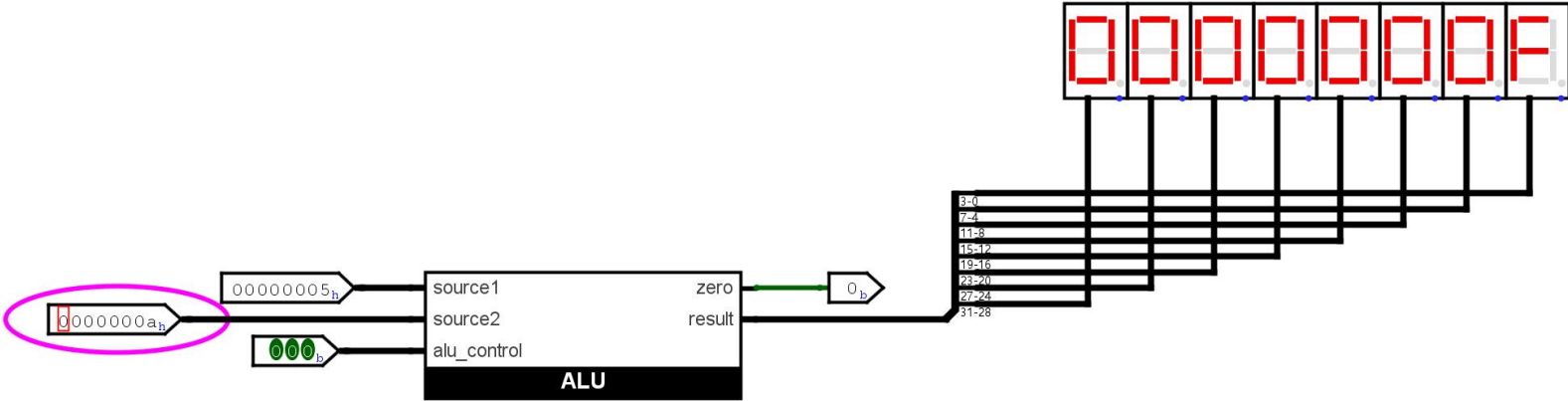
十六进制

复位值:

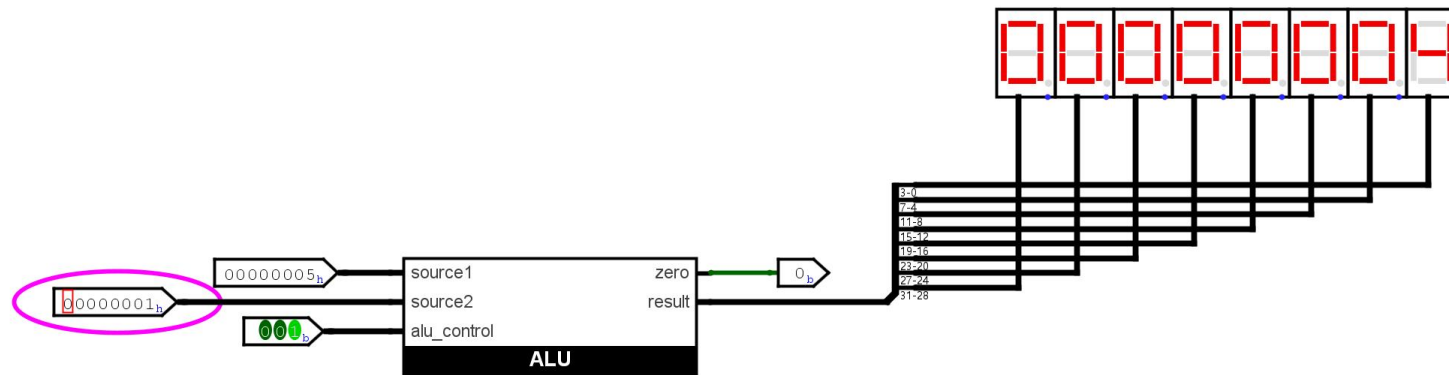
0x0

外观

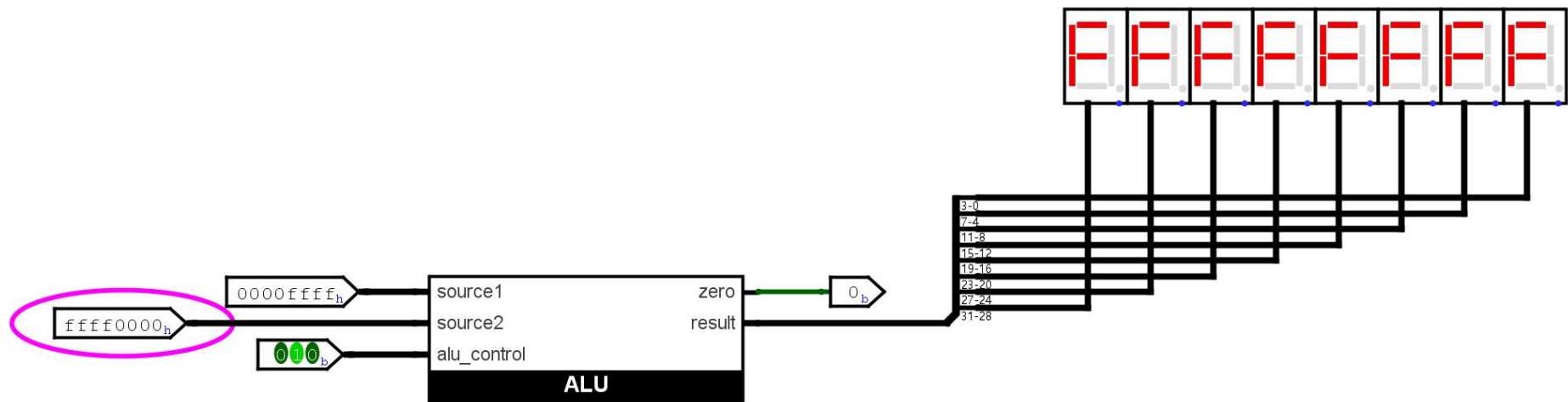
箭头形状



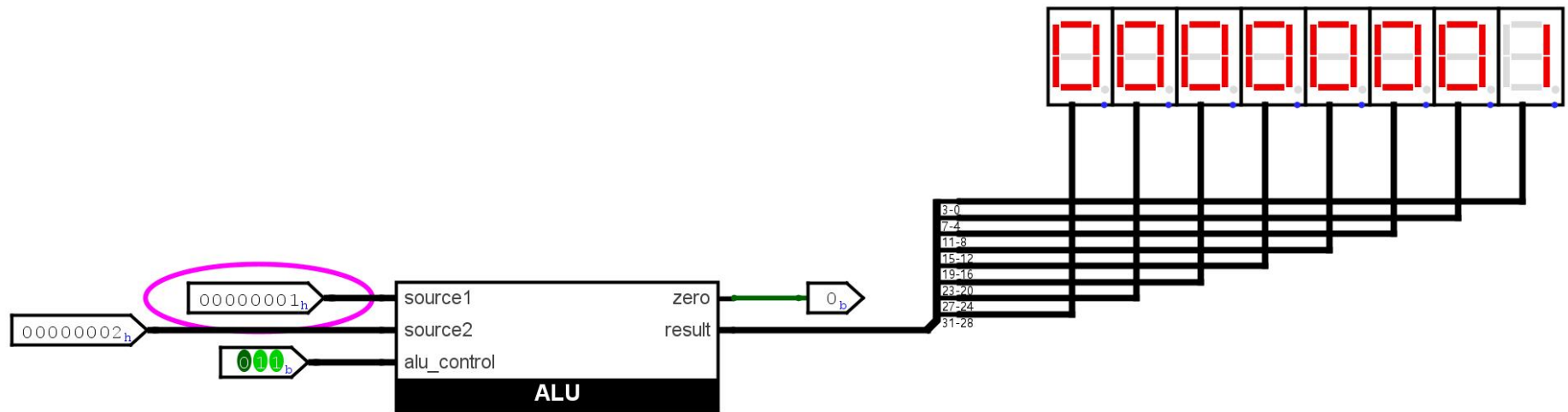
#	InputA	InputB	ALU_Op	Expected_Output
00000005	0000000A	000	0000000F	# Add
00000005	00000003	001	00000002	# Sub
0000FFFF	FFFF0000	010	FFFFFFFF	# OR
00000001	00000002	011	00000001	# SLT (1<2 is True)
00000000	00001234	100	12340000	# LUI



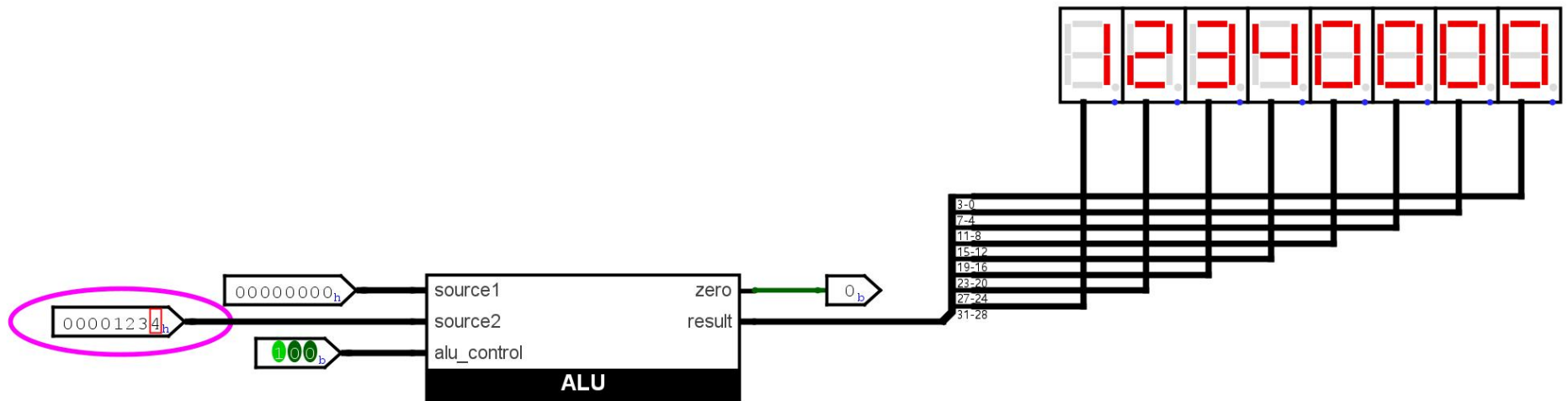
#	InputA	InputB	ALU_Op	Expected_Output
00000005	0000000A	000	0000000F	# Add
00000005	00000003	001	00000002	# Sub
0000FFFF	FFFF0000	010	FFFFFFFF	# OR
00000001	00000002	011	00000001	# SLT (1 < 2 is True)
00000000	00001234	100	12340000	# LUI



#	InputA	InputB	ALU_Op	Expected_Output
00000005	0000000A	000	0000000F	# Add
00000005	00000003	001	00000002	# Sub
0000FFFF	FFFF0000	010	FFFFFFFF	# OR
00000001	00000002	011	00000001	# SLT (1 < 2 is True)
00000000	00001234	100	12340000	# LUI



Plaintext				
#	InputA	InputB	ALU_Op	Expected_Output
00000005	0000000A	000	0000000F	# Add
00000005	00000003	001	00000002	# Sub
0000FFFF	FFFF0000	010	FFFFFFFF	# OR
00000001	00000002	011	00000001	# SLT (1<2 is True)
00000000	00001234	100	12340000	# LUI



Plaintext				
#	InputA	InputB	ALU_Op	Expected_Output
00000005	0000000A	000	0000000F	# Add
00000005	00000003	001	00000002	# Sub
0000FFFF	FFFF0000	010	FFFFFFFF	# OR
00000001	00000002	011	00000001	# SLT (1<2 is True)
00000000	00001234	100	12340000	# LUI